

AfriShield ML Evolution

A beginner's guide — from a hand-written formula to a real, tested improvement

This is the story of how AfriShield's flood-risk prediction went from a simple formula, to a machine-learning model trained on made-up data, to one trained on real historical floods and genuinely improved. No prior ML knowledge needed — every technical term is explained as it comes up, and defined again in the glossary at the end.

CHAPTER 1

The simple version (no ML at all)

Before any machine learning, there was just a formula a person wrote by hand: take the rainfall amount and the river level, average them together, and sort the result into "low," "medium," or "high" risk. Simple, and easy to explain to anyone — but it's just a fixed recipe. It doesn't "learn" anything; a human decided the rule in advance.

CHAPTER 2

Teaching a computer to guess (the first ML attempt)

We wanted a second opinion — something that learned a pattern by example instead of following a hand-written rule. But there was a problem: we didn't have real historical records of "it rained this much, the river was this high, and yes/no a flood happened." So we made up 2,000 fake example situations using code, decided what the "correct answer" should be for each one, and taught a simple model (**Logistic Regression** — see glossary) to recognize the pattern.

This worked, technically — the model got good at predicting the fake examples. But it never proved anything about the real world, because it never saw the real world.

CHAPTER 3

The moment of truth — checking against real floods

Later, we found a real historical record: the Dartmouth Flood Observatory, a genuine database of confirmed real floods going back decades. We used it to put both models to a real test — not "did they get the fake quiz right," but "would they have actually caught real floods that really happened?"

The honest result was humbling: on 239 real, confirmed flood-days, the simple hand-written formula caught **41%** of them (this is called **recall** — see glossary). The fancier trained ML model, trained on fake data, only caught **28%**. The "smarter-looking" approach was actually worse in the real world, because it had only ever practiced on made-up questions. A model is only as good as what it learned from, no matter how sophisticated the technique.

The real database gave us something new: **40,904 real days** of actual rainfall and river data, with **239 of those days** being real confirmed flood events. This time, instead of just using the real data to check the model afterward, we used it to actually teach the model from scratch.

One technical wrinkle: the real river data was measured in "discharge" (cubic meters of water flowing per second) — a different unit than the "river level in meters" our system actually uses. We couldn't directly convert one to the other, so we ranked each day's discharge against that same river's own history (e.g., "higher than 80% of all days ever seen on this river") and used that ranking as a stand-in for river level. It's an approximation, not a perfect conversion — and we say so honestly.

Another real wrinkle: floods are rare — only 239 of 40,904 days (about half a percent) were actual flood days. Left alone, a model could just learn to always guess "no flood" and still be right 99%+ of the time — technically accurate, but useless, since it would never warn anyone. The fix is a setting called **class weight balancing** (see glossary): it tells the model to pay extra attention to the rare flood examples instead of ignoring them.

The result, tested fairly on a chunk of real data the model never trained on (called **held-out test data** — see glossary): recall jumped to **81%** — three times better than the old fake-data model's 27%, and well ahead of the hand-written formula's 48% on that same test slice.

Catching more real floods came with a cost: the model also cried "flood!" on more days that were actually fine (a **false positive** — see glossary), raising the false-alarm rate from about 14% to about 34%. This is a completely normal, expected trade-off in machine learning — a model this alert can't also be perfectly picky.

The fix is called **threshold tuning** (see glossary). The model doesn't output a flat yes/no — it outputs a confidence number, like "I'm 73% sure this is a flood risk." Normally we just say "if it's over 50% confident, call it a risk." But that 50% cutoff is just a default, not a law of nature — we can slide that dial up or down and measure what happens at each setting, like a report card across the whole range.

We found something excellent: at a confidence cutoff of **80% instead of 50%**, the model catches **62.5%** of real floods (better than the formula's 48%) while also having a **lower** false-alarm rate (17.7% vs. the formula's 22%). Not a trade-off anymore — a real, straightforward improvement, on both counts, on real data.

The full journey, in one table

Version	Trained on	Recall (catches real floods)	False alarms
Hand-written formula	(no training, just math)	41–48%	22%
First ML model	fake, made-up data	27–28%	14%
New ML model (default cutoff)	real historical floods	81–87%	34–42%
New ML model (tuned cutoff)	real historical floods	62.5%	17.7%

The tuned version is the one worth being proud of — it's genuinely, honestly better than everything that came before it, measured against real events that really happened, not a made-up test.

Glossary

Training data	The example situations used to teach the model the pattern — like practice questions a student studies from.
Test data (a.k.a. "held-out" data)	Examples deliberately hidden from training, saved to check afterward whether the model actually learned something useful — or just memorized the practice questions. This is the only fair way to know if a model really works.
Recall	Out of all the real flood days that actually happened, what percentage did the model correctly flag? High recall means the model rarely misses a real flood.
False positive	A day the model flagged as risky that turned out to be perfectly safe — a false alarm.
False-positive rate (FPR)	Out of all the days that were actually safe, what percentage did the model incorrectly flag as risky? Lower is better — it means fewer unnecessary alarms.
Precision	Out of all the days the model flagged as risky, what percentage were real floods? Recall and precision are often in tension — being very cautious (high recall) usually means more false alarms (lower precision).
Threshold	The confidence cutoff used to turn a model's "I'm 73% sure" into a decision ("yes, warn" or "no, don't"). Sliding this cutoff up or down trades recall against false alarms.
Class imbalance	When one outcome (like "no flood") is far more common in the data than another (like "flood"), making it easy for a lazy model to just always guess the common one and still look accurate.
Class weight balancing	A training setting that tells the model to pay extra attention to the rare, important category (floods) instead of letting it get ignored because there's fewer examples of it.
Logistic Regression	A simple, well-understood method for sorting examples into categories by finding the best dividing line between them. Chosen here because it's easy to explain, not because it's the most powerful option available.
Overfitting	When a model gets very good at its exact practice examples but fails on new, real situations — closely related to Chapter 2's lesson: performing well on fake data proves nothing about the real world.
Youden's J statistic	A simple way to combine "how many did we catch" (recall) and "how many false alarms" (false-positive rate) into one number, used to help compare different threshold settings at a glance.